

VideoGame Platform

Summary

VideoGame Platform is a web platform for video-game discovery, tracking, ratings, and analysis. It is conceptually inspired by services such as FilmAffinity but is specialized in video games.

The platform will let users browse games, learn about recent and upcoming releases, discover titles based on their preferences, and access aggregated information from several sources.

In addition to being a functional product, the project will be developed as an advanced software-architecture learning environment. It will be designed as if it were a real product maintained by an organization with different teams, professional roles, development processes, business needs, and technical constraints.

Primary purpose

The project's main objective is to improve my capabilities as a solution architect and technical leader through the progressive design and construction of a complete platform.

The project should let me practise:

- Digital product design.
- Software architecture.
- Domain-Driven Design.
- Hexagonal architecture.
- Distributed systems and microservices.
- API design.
- Integration with external systems.
- Asynchronous processing and event-driven architectures.
- Persistence and data modelling.
- Security and identity management.
- Observability.
- DevOps and cloud-native platforms.
- Software quality and automated testing.
- Scalability, resilience, and performance.
- Technical team leadership.
- Architectural documentation and decision-making.
- Product evolution from an MVP to a large-scale platform.

Architectural learning takes priority, but decisions must not introduce artificial complexity. The platform should use technologies and patterns only when they address a real need or an explicit learning objective.

Product vision

VideoGame Platform should become a central place where players can:

- Browse detailed information about video games.
- Discover each week's new releases.
- Review upcoming releases.
- Rate video games.
- Write reviews.
- Create personal lists.
- Track games as pending, started, completed, or abandoned.
- Follow franchises, studios, platforms, or genres.
- Receive personalized recommendations.
- Compare ratings from users and professional media.
- Read professional reviews.
- View external scores such as Metacritic, OpenCritic, or other authorized sources.
- Explore trends, popularity, and statistics.
- Participate in a community of players.

Primary users

Visitor

Can search for video games and browse public information, ratings, reviews, releases, and public lists.

Registered user

Can rate video games, write reviews, create lists, follow titles, and maintain a personal library.

Moderator

Manages reported content, reviews, users, and conflicts within the community.

Content administrator

Can correct game records, manage external sources, and supervise data quality.

Platform administrator

Manages configuration, permissions, integrations, operations, and overall platform behaviour.

Functional capabilities

Video-game catalogue

Each game may include:

- Title and alternative titles.
- Description.
- Cover, screenshots, and videos.
- Release date.
- Platforms.
- Genres.
- Game modes.
- Developers and publishers.
- Franchise or series.
- Age rating.
- Available languages.
- Technical requirements.
- Editions.
- Downloadable content and expansions.
- Relationships with other video games.

Releases and news

The platform will show:

- This week's new releases.
- Upcoming releases.
- Calendar by platform.
- Featured releases.
- Delays and date changes.
- New expansions and major updates.

Ratings

Users will be able to:

- Rate video games.
- Change or delete their rating.
- Write reviews.
- Rate other users' reviews.
- View the rating distribution.
- Compare user ratings with ratings from professional media.

External ratings must be integrated in accordance with each provider's licence, terms of use, and permitted access mechanisms. Unauthorized scraping will be avoided.

Personal library

Users will be able to classify their games as:

- Pending.
- Playing.
- Completed.
- Abandoned.
- Favourite.

They will also be able to record platform, hours played, start date, completion date, and personal notes.

Lists

Users will be able to create public or private lists, for example:

- Best games of the year.
- Upcoming games I want to buy.
- Best RPGs.
- Games to complete.
- Recommendations for cooperative play.

Discovery and recommendations

The platform may recommend video games based on:

- Preferred genres.
- Rating history.
- Personal library.
- Available platforms.
- Similar games.
- Popularity.
- Trends.
- Affinity with other users.

Early versions may use simple rules. Advanced recommendation systems will be introduced when enough data and a real need exist.

Non-functional requirements

The system should evolve to support:

- High availability.
- Horizontal scalability.
- Data consistency and quality.
- Operational traceability.
- Security by default.
- Personal-data protection.
- Predictable performance.
- Resilience to external failures.
- Complete observability.
- Automated deployments.
- Independent component evolution.
- Backward compatibility for APIs and events.
- Controlled operating costs.

Development approach

The project will be developed through vertical slices. Each increment should deliver a complete functional capability from the user interface through persistence and operations.

Indicative evolution

- 1. Basic video-game catalogue.
- 2. Search and game-page browsing.
- 3. User registration and identity.
- 4. Ratings and reviews.
- 5. Personal library.
- 6. Releases and news.
- 7. External integrations.
- 8. Lists and community.
- 9. Recommendations.
- 10. Analytics, moderation, and advanced capabilities.

The system will not be split into microservices prematurely. It may start as a modular monolith or a small number of clearly bounded services, extracting components only when technical, organizational, or scalability reasons exist.

Simulated organization

The project will be treated as if these roles participated:

- Product Manager.
- Product Owner.
- Solution Architect.
- Tech Lead.
- Backend Engineers.
- Frontend Engineers.
- UX/UI Designer.
- QA Engineer.
- Platform Engineer.
- DevOps Engineer.
- Site Reliability Engineer.
- Security Engineer.
- Data Engineer.
- Data Analyst.
- Content Moderator.
- Compliance and Privacy Lead.

Decisions should consider the needs and perspectives of these roles even though the project is developed by one person.

Principles

- Solve the product problem first.
- Prefer familiar, mature, and maintainable technologies.
- Avoid accidental complexity.
- Do not use microservices by default.
- Design explicit domain boundaries.
- Document important decisions.
- Automate tests, builds, and deployments.
- Measure before optimizing.
- Design for evolution rather than predicting the entire future.
- Treat security, observability, and operations as part of the product.
- Introduce new technologies only when they add value or relevant learning.
- Maintain a coherent and understandable architecture.